

Entregable Proyecto no. 2 Animatrónico

Universidad del Valle de Guatemala

Grettel María Escobedo Mendóza, Carné no. 23586

May 19, 2026

1 Proyecto Funcionamiento

Para la realización de este proyecto se tomará como base la propuesta llamada Rostro animatrónico.

Este proyecto consiste en la fabricación de un animatrónico que pueda duplicar el funcionamiento de los ojos humanos con movimientos fieles a la realidad. Este movimiento ocular está dividido en dos ejes principales, el cual creará la ilusión de ojos con pupilas que pueden ver en dirección de derecha a izquierda, arriba y abajo. Esto será realizado con ayuda de tres diferentes ejes impresos en PLA, los cuales ayudarán a ajustar los servomotores a su respectivo eje de rotación.

Aclaración y modificaciones

Sin embargo, se realizarán cambios en la disponibilidad de los servomotores que están destinados a los gestos del movimiento de las cejas. Se cambiarán estos últimos, por servomotores que estén encargados de realizar un movimiento igual al de un parpado móvil sobre los ojos del animatrónico.

Esta decisión se toma por preferencias estéticas, debido a que se busca igualar el rostro de un animal, en los cuales, las cejas usualmente pasan desapercibidos. Inicialmente las cejas presentan un movimiento de rotación paralelo al rostro, del animatrónico, sin embargo, con la modificación de los parpados, los servomotores se moverán con su eje de rotación perpendicular al plano del rostro. De esta manera seguirán la trayectoria curva de los ojos.

Son 5 modos específicos en los cuales se reparte el funcionamiento del físico. (UART- manual-EEPROM- ADAFRUIT IO)

1.1 Modo de configuración Normal

Se utiliza la lectura ADC para leer los potenciómetros cuando estos son manejados desde el protoboard. Una vez leído el valor el Potenciómetro rota a la posición deseada la aguja o brazo del servomotor.

Se habilitó la lectura a través de PC2 hasta PC5, conjuntamente con A6 Y A7.

1.2 Modo de configuración EEPROM

En esta parte se aprovecha la memoria del Nano, se implementan cinco posiciones diferentes, cada posición posee una combinación diferentes a cada motor. El objetivo es seleccionar una combinación y se crea una gesticulación en el animatrónico en específico.

Estas posiciones son editables, pues por medio de un botón en Puerto C, pin 1, se crea una interrupción, la cual, variará en mi bandera a 1.

Una vez realizada la interrupción:

Se leerá la posición actual de los servomotores en el protoboard o en el dashboard de adafruit (que deberían ser iguales) y posteriormente guarda este valor en EEPROM.

Adicionalmente, para editar el resto de las localidades 0- 23 se utiliza la opción 5 del menú, (edición de poses), de la mano de la librería PosesEDITOR.

1.3 Modo de configuración Adafruit IO

Este modo se utiliza para realizar la comunicación entre el usuario y los servomotores, asimismo, la comunicación entre el modo manual y la gráfica de estado en adafruit.

Hace utilización de la comunicación Serial y se configuró desde la terminal de Phyton. Posteriormente la información obtenida se manda al dashboard de la aplicación Adafruit.

1.4 Lectura ADC

Este, pese a no ser un modo de funcionamiento como tal, siempre se encuentra presente para la lectura ADC, de los potenciómetros.

1.5 Configuración UART

Se implementó al ser la comunicación entre la terminal de microchip studio y el usuario, Mediante esta interfaz del data visualizer, se manda el menú de opciones con acciones distintas.

- Movimiento de motores de ojos
- Movimiento de motores de los parpados
- lectura de posiciones en EEPROM
- Modo manual adc
- Editar posiciones en EEPROM

Cada opción hace que se ejecute una nuevo menú de opciones en las cuales se puede determinar el motor en específico y su grado de rotación, en la número 3 se despliega un menú para leer y colocar las posiciones de eeprom ya guardadas y en el modo manual se edita con los potenciómetros el pulso.

En la última opción se utiliza un submenú para desplegar las cinco posiciones predeterminadas para las posiciones y agregar una posición nueva de edición por el gusto del usuario.

Tabla de orden y distribución:

Tabla 1: Asignación de servomotores controlados por UART

Comando UART	Servo	Timer	Pin físico ATmega328P	Pin Arduino
I1:90;	Ojo izquierdo base	Timer1 OC1A	PB1	D9
D1:90;	Ojo derecho base	Timer1 OC1B	PB2	D10
I2:90;	Pupila izquierda	Timer0 software PWM	PD6	D6
D2:90;	Pupila derecha	Timer0 software PWM	PD5	D5
M1:90;	Párpado 1	Timer2 software PWM	PD3	D3
M2:90;	Párpado 2	Timer2 software PWM	PB3	D11

Distribución según posiciones de potenciómetro y color de cable del pin de salida:

Tabla 2: Asignación de potenciómetros y servomotores

Potenciómetro	Motor	Función	Timer	Color
Primer potenciómetro	I1	Base ojo izquierdo	Timer1	Azul 2
Segundo potenciómetro	D1	Base ojo derecho	Timer1	Azul 1
Tercer potenciómetro	I2	Pupila izquierda	Timer0	Amarillo 1
Cuarto potenciómetro	D2	Pupila derecha	Timer0	Amarillo 2
Quinto potenciómetro	M1	Párpado 1	Timer2	Rojo 1
Sexto potenciómetro	M2	Párpado 2	Timer2	Rojo 2

2 Maquina de Estados

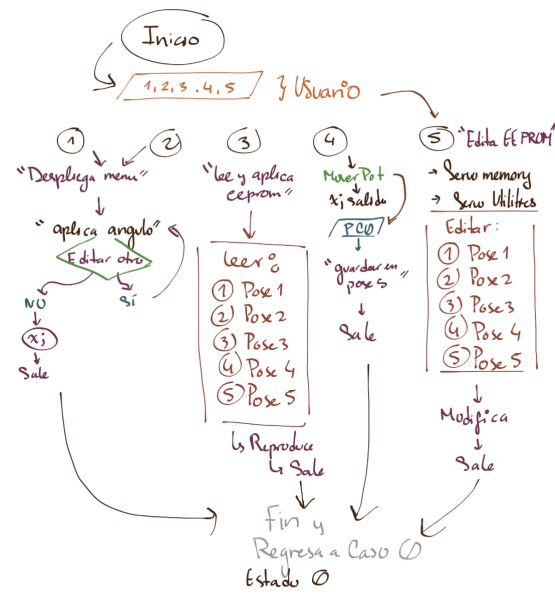


Figura 1: Máquina de Estados

2.1 Diagrama de flujo

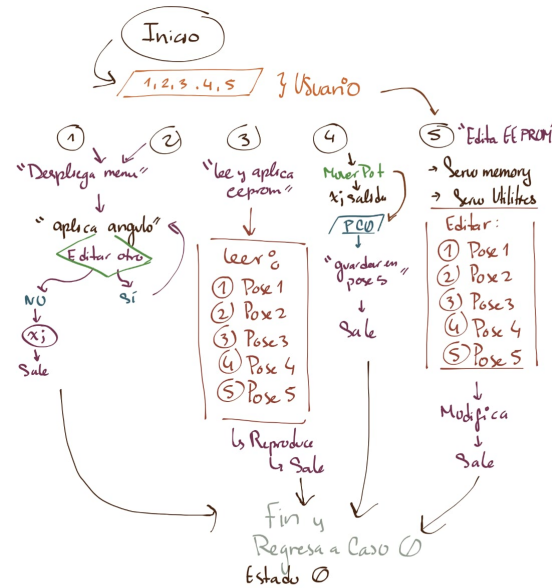


Figura 2: Diagrama de flujo

3 Cálculos

Se realizaron los siguientes cálculos:

3.1 Cálculo de PWM para los Timers Utilizados

3.1.1 Timer1 - PWM por Hardware

Para el Timer1 se utilizó el modo Fast PWM con TOP definido por el registro *ICR1*.

La fórmula utilizada para la frecuencia fue:

$$f_{PWM} = \frac{f_{clk}}{Prescaler \times (1 + TOP)}$$

Donde:

$$f_{clk} = 16MHz$$

$$Prescaler = 8$$

$$TOP = ICR1 = 39999$$

Sustituyendo:

$$f_{PWM} = \frac{16000000}{8(1 + 39999)}$$

$$f_{PWM} = \frac{16000000}{320000}$$

$$f_{PWM} = 50Hz$$

Por lo tanto, el período de la señal PWM es:

$$T = \frac{1}{f}$$

$$T = \frac{1}{50}$$

$$T = 20ms$$

El temporizador trabaja a:

$$f_{timer} = \frac{16MHz}{8}$$

$$f_{timer} = 2MHz$$

Por consiguiente:

$$T_{tick} = \frac{1}{2MHz}$$

$$T_{tick} = 0.5\mu s$$

3.1.2 Cálculo de OCR1A y OCR1B

Para obtener el ancho de pulso se utiliza:

$$OCR = \frac{T_{pulso}}{T_{tick}}$$

Para $1ms$:

$$OCR = \frac{1000\mu s}{0.5\mu s}$$

$$OCR = 2000$$

Para $1.5ms$:

$$OCR = \frac{1500\mu s}{0.5\mu s}$$

$$OCR = 3000$$

Para $2ms$:

$$OCR = \frac{2000\mu s}{0.5\mu s}$$

$$OCR = 4000$$

Por lo tanto:

$$OCR1A, OCR1B \in [2000, 4000]$$

corresponden al rango angular aproximado de:

$$0^\circ \rightarrow 180^\circ$$

3.1.3 Timer0 - PWM por Software

Para Timer0 se utilizó el modo CTC.

La frecuencia interna del timer es:

$$f_{timer} = \frac{16MHz}{8}$$

$$f_{timer} = 2MHz$$

Cada tick equivale a:

$$T_{tick} = 0.5\mu s$$

Se configuró:

$$OCR0A = 39$$

Entonces el tiempo entre interrupciones es:

$$T_{int} = (OCR0A + 1)(T_{tick})$$

$$T_{int} = (39 + 1)(0.5\mu s)$$

$$T_{int} = 20\mu s$$

El contador se reinicia cada:

1000 interrupciones

Por lo tanto:

$$T_{PWM} = 1000(20\mu s)$$

$$T_{PWM} = 20ms$$

Y finalmente:

$$f_{PWM} = \frac{1}{20ms}$$

$$f_{PWM} = 50Hz$$

3.1.4 Cálculo de servo0A_ticks y servo0B_ticks

Para PWM por software:

$$Ticks = \frac{T_{pulso}}{T_{int}}$$

Para 1ms:

$$Ticks = \frac{1000\mu s}{20\mu s}$$

$$Ticks = 50$$

Para 1.5ms:

$$Ticks = \frac{1500\mu s}{20\mu s}$$

$$Ticks = 75$$

Para $2ms$:

$$Ticks = \frac{2000\mu s}{20\mu s}$$

$$Ticks = 100$$

Por lo tanto:

$$servo0A_ticks, servo0B_ticks \in [50, 100]$$

3.1.5 Timer2 - PWM por Software

El Timer2 utiliza exactamente la misma configuración que Timer0:

$$Prescaler = 8$$

$$OCR2A = 39$$

$$f_{timer} = 2MHz$$

$$T_{tick} = 0.5\mu s$$

Tiempo entre interrupciones:

$$T_{int} = (39 + 1)(0.5\mu s)$$

$$T_{int} = 20\mu s$$

El frame completo es:

$$1000(20\mu s)$$

$$= 20ms$$

Por lo tanto:

$$f_{PWM} = 50Hz$$

3.1.6 Cálculo de servo2A_ticks y servo2B_ticks

Para $1ms$:

$$Ticks = \frac{1000\mu s}{20\mu s}$$

$$Ticks = 50$$

Para $1.5ms$:

$$Ticks = \frac{1500\mu s}{20\mu s}$$

$$Ticks = 75$$

Para $2ms$:

$$Ticks = \frac{2000\mu s}{20\mu s}$$

$$Ticks = 100$$

Por lo tanto:

$$servo2A_ticks, servo2B_ticks \in [50, 100]$$

3.1.7 Fórmula de PWM para Timer1

Para el Timer1 en modo Fast PWM:

$$f_{PWM} = \frac{f_{clk}}{Prescaler \times (1 + TOP)}$$

En este proyecto:

$$f_{clk} = 16MHz$$

$$Prescaler = 8$$

$$TOP = ICR1 = 39999$$

Entonces:

$$f_{PWM} = \frac{16000000}{8(1 + 39999)}$$

$$f_{PWM} = \frac{16000000}{320000}$$

$$f_{PWM} = 50Hz$$

Para PWM Phase Correct en Timer1:

$$f_{PWM} = \frac{f_{clk}}{2 \times Prescaler \times TOP}$$

3.2 Cálculo del Baudrate UART

Para la comunicación serial UART se configuró un baudrate de:

$$BAUD = 9600$$

La frecuencia del microcontrolador es:

$$f_{osc} = 16MHz$$

Para modo asíncrono normal se utiliza la fórmula:

$$UBRRn = \frac{f_{osc}}{16 \times BAUD} - 1$$

Sustituyendo valores:

$$UBRRn = \frac{16,000,000}{16 \times 9600} - 1$$

$$UBRRn = \frac{16,000,000}{153600} - 1$$

$$UBRRn = 104.16 - 1$$

$$UBRRn \approx 103$$

Por lo tanto, en el código se configuró:

$$UBRR0 = 103$$

Para verificar el baudrate real obtenido:

$$BAUD_{real} = \frac{f_{osc}}{16 \times (UBRRn + 1)}$$

Sustituyendo:

$$BAUD_{real} = \frac{16,000,000}{16 \times (103 + 1)}$$

$$BAUD_{real} = \frac{16,000,000}{1664}$$

$$BAUD_{real} \approx 9615$$

El porcentaje de error se calcula mediante:

$$Error\% = \left(\frac{BAUD_{real}}{BAUD} - 1 \right) \times 100$$

$$Error\% = \left(\frac{9615}{9600} - 1 \right) \times 100$$

$$Error\% \approx 0.16\%$$

El error obtenido es menor al 2%, por lo que la comunicación serial es válida y estable según la tablita.

4 Dashboard

Se conectaron los 6 motores a los medidores de "Gauge", presentes en el dashboard del lado izquierdo. Estos permiten realizar la lectura de los grados de inclinación del servomotor.

Mientras tanto, en la parte derecha del panel, se implementaron los deslizantes, encargados de ser la entrada para que el usuario maneje cada servomotor.

Sin embargo, se utilizó únicamente, 4 deslizantes para mandar la información, siendo estos los correspondientes a los ojos y sus bases. Esto debido a la limitante de los 10 feeds disponibles para el proyecto.



Figura 3: Imagen. Dashboard

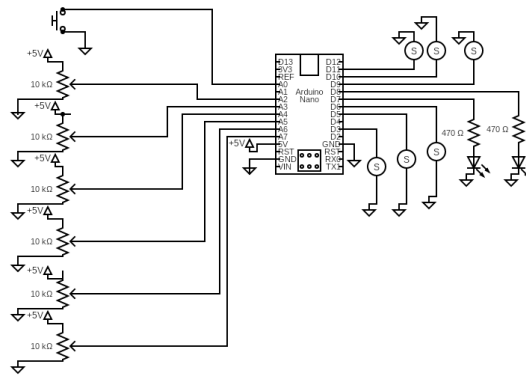


Figura 4: Esquemático Circuito

5 Entregables físicos

5.1 Código de colores

Tabla 3: Asignación de colores en el cableado

Color	Asignación
Blanco	Salidas de Leds de estado
Rojo	Color de alimentación 5V y servomotores con Timer2
Negro	Negativo (GND)
Amarillo	Lectura ADC pot y servomotor con Timer 0
Verde	Entrada lógica de PC0
Azul	Salida lógica servo Timer 1

5.2 Esquemático

[H]

5.3 Protoboard

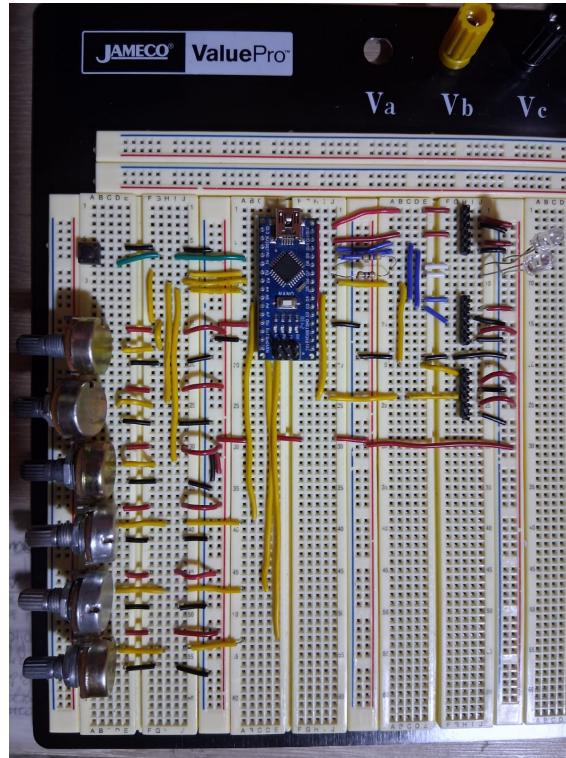


Figura 5: Implementación a Protoboard

Tabla 4: Lista de materiales

Material	Asignación
Cartulina	Carcasa o cara del animatrónico
Cartón	Estructura y soporte del animatrónico
PLA	Anillo para giro de ojos
Tornillos M5	Ejes de rotación de los ojos
Pelotas de pinpong	ojos

6 Links

Link del video

https://uvgt-my.sharepoint.com/:f:/g/personal/esc23586_uvg_edu_gt/IgAG0BzEtBX8Q7iIw7qBqs_nAajEQjIw_UhOyC75RxHvxc?e=L6TQU7

Link del repositorio

https://github.com/esc23586/23586_C_PROYECTO2

Link del tablero de Adafruit

<https://io.adafruit.com/Grett2026/dashboards/prueba-dashboard>

7 Anexos

Link del tablero de Adafruit

<https://www.youtube.com/watch?v=845gXvtQFLg&t=6s>